

WebSocket Chat Room - Complete Guide with Room Management

Understanding How WebSocket "message" Event Works

The Core Concept

When you call `ws.send()` from the client, the WebSocket protocol automatically:

1. Wraps your data in a WebSocket frame
2. Sends it over the TCP connection
3. The server's WebSocket library automatically detects incoming frames
4. Triggers the `'message'` event on the server

You don't manually specify the event type - the WebSocket protocol handles this automatically!

```
javascript

// CLIENT SIDE
ws.send('Hello'); // Automatically creates a "message" frame

// SERVER SIDE
ws.on('message', (data) => { // Automatically triggered when frame arrives
  console.log(data); // 'Hello'
});
```

How It Works Under the Hood



The Solution: Message Structure with JSON

Since WebSocket only has one `'message'` event, we need to structure our messages with types:

javascript

```
// Instead of just sending text
ws.send('Hello');

// We send structured JSON with a type
ws.send(JSON.stringify({
  type: 'chat',
  message: 'Hello'
}));

// Or
ws.send(JSON.stringify({
  type: 'join-room',
  room: 'general'
}));
```

Complete Chat Room Implementation

Project Structure

```
chat-room/
├── server.js      # WebSocket server
├── public/
│   ├── index.html # Chat UI
│   ├── app.js     # Client WebSocket logic
│   └── style.css   # Styling
└── package.json
```

1. Server Implementation (server.js)

javascript

```
const WebSocket = require('ws');
const http = require('http');
const express = require('express');
const path = require('path');

const app = express();
const server = http.createServer(app);
const wss = new WebSocket.Server({ server });

// Serve static files
app.use(express.static('public'));

// Store clients and rooms
const clients = new Map(); // Map<WebSocket, ClientInfo>
const rooms = new Map(); // Map<roomName, Set<WebSocket>>

// Initialize some default rooms
rooms.set('general', new Set());
rooms.set('random', new Set());
rooms.set('tech', new Set());

console.log('WebSocket Chat Server Starting...');

wss.on('connection', (ws, request) => {
  console.log('New client connected from:', request.socket.remoteAddress);

  // Initialize client info
  const clientInfo = {
    username: null,
    currentRoom: null,
    ws: ws
  };

  clients.set(ws, clientInfo);

  // Send welcome message
  ws.send(JSON.stringify({
    type: 'connection',
    message: 'Connected to chat server',
    availableRooms: Array.from(rooms.keys())
  }));

  // =====
  // MESSAGE EVENT HANDLER - This is where ALL client messages arrive
  // =====
  ws.on('message', (data) => {
```

```
try {  
  // Parse the incoming message  
  const message = JSON.parse(data.toString());  
  
  console.log('Received message:', message);  
  
  // Route based on message type  
  switch (message.type) {  
    case 'set-username':  
      handleSetUsername(ws, message);  
      break;  
  
    case 'join-room':  
      handleJoinRoom(ws, message);  
      break;  
  
    case 'leave-room':  
      handleLeaveRoom(ws, message);  
      break;  
  
    case 'chat':  
      handleChatMessage(ws, message);  
      break;  
  
    case 'create-room':  
      handleCreateRoom(ws, message);  
      break;  
  
    case 'private-message':  
      handlePrivateMessage(ws, message);  
      break;  
  
    case 'typing':  
      handleTyping(ws, message);  
      break;  
  
    default:  
      ws.send(JSON.stringify({  
        type: 'error',  
        message: 'Unknown message type'  
      }));  
  }  
  
} catch (error) {  
  console.error('Error parsing message:', error);  
  ws.send(JSON.stringify({  
    type: 'error',
```

```
        message: 'Invalid message format'
    }));
    });

// Handle client disconnect
ws.on('close', () => {
    handleDisconnect(ws);
});

// Handle errors
ws.on('error', (error) => {
    console.error('WebSocket error:', error);
});
});

// =====
// MESSAGE HANDLERS
// =====

function handleSetUsername(ws, message) {
    const clientInfo = clients.get(ws);
    const username = message.username.trim();

    if (!username) {
        ws.send(JSON.stringify({
            type: 'error',
            message: 'Username cannot be empty'
        }));
        return;
    }

    // Check if username is taken
    const isTaken = Array.from(clients.values()).some(
        client => client.username === username && client.ws !== ws
    );

    if (isTaken) {
        ws.send(JSON.stringify({
            type: 'error',
            message: 'Username already taken'
        }));
        return;
    }

    clientInfo.username = username;
```

```
ws.send(JSON.stringify({
  type: 'username-set',
  username: username,
  message: `Username set to ${username}`
}));

console.log(`Client set username to: ${username}`);
}

function handleJoinRoom(ws, message) {
  const clientInfo = clients.get(ws);
  const roomName = message.room;

  if (!clientInfo.username) {
    ws.send(JSON.stringify({
      type: 'error',
      message: 'Please set a username first'
    }));
    return;
  }

  // Leave current room if in one
  if (clientInfo.currentRoom) {
    handleLeaveRoom(ws, { room: clientInfo.currentRoom });
  }

  // Create room if it doesn't exist
  if (!rooms.has(roomName)) {
    rooms.set(roomName, new Set());
  }

  // Add client to room
  rooms.get(roomName).add(ws);
  clientInfo.currentRoom = roomName;

  // Notify client they joined
  ws.send(JSON.stringify({
    type: 'joined-room',
    room: roomName,
    message: `You joined room: ${roomName}`
  }));

  // Notify others in the room
  broadcastToRoom(roomName, {
    type: 'user-joined',
    username: clientInfo.username,
    room: roomName,
```

```
    message: `${clientInfo.username} joined the room`,
    timestamp: new Date().toISOString()
  }, ws); // Exclude the joining user

// Send room members list
const members = getRoomMembers(roomName);
ws.send(JSON.stringify({
  type: 'room-members',
  room: roomName,
  members: members
}));

console.log(`${clientInfo.username} joined room: ${roomName}`);
}

function handleLeaveRoom(ws, message) {
  const clientInfo = clients.get(ws);
  const roomName = message.room || clientInfo.currentRoom;

  if (!roomName || !rooms.has(roomName)) {
    return;
  }

  // Remove from room
  rooms.get(roomName).delete(ws);

  // Notify others
  if (clientInfo.username) {
    broadcastToRoom(roomName, {
      type: 'user-left',
      username: clientInfo.username,
      room: roomName,
      message: `${clientInfo.username} left the room`,
      timestamp: new Date().toISOString()
    });
  }

  clientInfo.currentRoom = null;

  ws.send(JSON.stringify({
    type: 'left-room',
    room: roomName,
    message: `You left room: ${roomName}`
  }));

  console.log(`${clientInfo.username} left room: ${roomName}`);
}
```

```
function handleChatMessage(ws, message) {
  const clientInfo = clients.get(ws);

  if (!clientInfo.username) {
    ws.send(JSON.stringify({
      type: 'error',
      message: 'Please set a username first'
    }));
    return;
  }

  if (!clientInfo.currentRoom) {
    ws.send(JSON.stringify({
      type: 'error',
      message: 'Please join a room first'
    }));
    return;
  }

  const chatMessage = {
    type: 'chat',
    username: clientInfo.username,
    message: message.message,
    room: clientInfo.currentRoom,
    timestamp: new Date().toISOString()
  };

  // Broadcast to everyone in the room (including sender)
  broadcastToRoom(clientInfo.currentRoom, chatMessage);

  console.log(`[${clientInfo.currentRoom}] ${clientInfo.username}: ${message.message}`);
}

function handleCreateRoom(ws, message) {
  const clientInfo = clients.get(ws);
  const roomName = message.room.trim();

  if (!roomName) {
    ws.send(JSON.stringify({
      type: 'error',
      message: 'Room name cannot be empty'
    }));
    return;
  }

  if (rooms.has(roomName)) {
```



```
ws.send(JSON.stringify({
  type: 'error',
  message: 'Room already exists'
}));
return;
}

// Create new room
rooms.set(roomName, new Set());

// Notify all clients about new room
broadcast({
  type: 'room-created',
  room: roomName,
  creator: clientInfo.username,
  message: `New room created: ${roomName}`
});

console.log(`Room created: ${roomName} by ${clientInfo.username}`);
}
```

```
function handlePrivateMessage(ws, message) {
  const clientInfo = clients.get(ws);
  const targetUsername = message.to;

  // Find target client
  let targetWs = null;
  for (const [socket, info] of clients.entries()) {
    if (info.username === targetUsername) {
      targetWs = socket;
      break;
    }
  }

  if (!targetWs) {
    ws.send(JSON.stringify({
      type: 'error',
      message: `User ${targetUsername} not found`
    }));
    return;
  }
}
```

```
const privateMessage = {
  type: 'private-message',
  from: clientInfo.username,
  message: message.message,
  timestamp: new Date().toISOString()
}
```

```

};

// Send to recipient
targetWs.send(JSON.stringify(privateMessage));

// Send confirmation to sender
ws.send(JSON.stringify({
  type: 'private-message-sent',
  to: targetUsername,
  message: message.message,
  timestamp: new Date().toISOString()
}));
}

function handleTyping(ws, message) {
  const clientInfo = clients.get(ws);

  if (!clientInfo.currentRoom) return;

  broadcastToRoom(clientInfo.currentRoom, {
    type: 'typing',
    username: clientInfo.username,
    isTyping: message.isTyping
  }, ws);
}

function handleDisconnect(ws) {
  const clientInfo = clients.get(ws);

  if (clientInfo) {
    // Leave current room
    if (clientInfo.currentRoom) {
      handleLeaveRoom(ws, { room: clientInfo.currentRoom });
    }

    console.log(`Client disconnected: ${clientInfo.username || 'Unknown'}`);
  }

  clients.delete(ws);
}

// =====
// HELPER FUNCTIONS
// =====

function broadcastToRoom(roomName, message, excludeWs = null) {
  if (!rooms.has(roomName)) return;

```

```
const messageStr = JSON.stringify(message);

rooms.get(roomName).forEach((clientWs) => {
  if (clientWs !== excludeWs && clientWs.readyState === WebSocket.OPEN) {
    clientWs.send(messageStr);
  }
});
}

function broadcast(message, excludeWs = null) {
  const messageStr = JSON.stringify(message);

  clients.forEach((clientInfo, clientWs) => {
    if (clientWs !== excludeWs && clientWs.readyState === WebSocket.OPEN) {
      clientWs.send(messageStr);
    }
  });
}

function getRoomMembers(roomName) {
  if (!rooms.has(roomName)) return [];

  const members = [];
  rooms.get(roomName).forEach((clientWs) => {
    const clientInfo = clients.get(clientWs);
    if (clientInfo && clientInfo.username) {
      members.push(clientInfo.username);
    }
  });

  return members;
}

// Start server
const PORT = process.env.PORT || 3000;
server.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
  console.log(`WebSocket server ready on ws://localhost:${PORT}`);
});

// Graceful shutdown
process.on('SIGTERM', () => {
  console.log('SIGTERM received, closing server...');
  server.close(() => {
    console.log('Server closed');
    process.exit(0);
  });
});
```

```
});
```

```
});
```

2. Client Implementation (public/app.js)

```
javascript
```

```

let ws = null;
let currentUsername = null;
let currentRoom = null;
let reconnectAttempts = 0;
const maxReconnectAttempts = 5;

// DOM Elements
const usernameSection = document.getElementById('username-section');
const chatSection = document.getElementById('chat-section');
const usernameInput = document.getElementById('username-input');
const setUsernameBtn = document.getElementById('set-username-btn');
const roomsList = document.getElementById('rooms-list');
const currentRoomName = document.getElementById('current-room-name');
const membersList = document.getElementById('members-list');
const messagesContainer = document.getElementById('messages');
const messageInput = document.getElementById('message-input');
const sendBtn = document.getElementById('send-btn');
const createRoomBtn = document.getElementById('create-room-btn');
const connectionStatus = document.getElementById('connection-status');

// Initialize WebSocket connection
function connect() {
  const protocol = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
  const wsUrl = `${protocol}://${window.location.host}`;

  ws = new WebSocket(wsUrl);

  // =====
  // CONNECTION OPENED
  // =====

  ws.addEventListener('open', (event) => {
    console.log('Connected to WebSocket server');
    updateConnectionStatus('connected');
    reconnectAttempts = 0;
  });

  // =====
  // MESSAGE RECEIVED - This is where ALL server messages arrive
  // =====

  ws.addEventListener('message', (event) => {
    try {
      const data = JSON.parse(event.data);
      console.log('Received message:', data);

      // Route based on message type
      switch (data.type) {

```

```
case 'connection':
  handleConnection(data);
  break;

case 'username-set':
  handleUsernameSet(data);
  break;

case 'joined-room':
  handleJoinedRoom(data);
  break;

case 'left-room':
  handleLeftRoom(data);
  break;

case 'chat':
  handleChatMessage(data);
  break;

case 'user-joined':
  handleUserJoined(data);
  break;

case 'user-left':
  handleUserLeft(data);
  break;

case 'room-members':
  handleRoomMembers(data);
  break;

case 'room-created':
  handleRoomCreated(data);
  break;

case 'private-message':
  handlePrivateMessage(data);
  break;

case 'private-message-sent':
  handlePrivateMessageSent(data);
  break;

case 'typing':
  handleTyping(data);
  break;
```

```

    case 'error':
        handleError(data);
        break;

    default:
        console.warn('Unknown message type:', data.type);
    }

} catch (error) {
    console.error('Error parsing message:', error);
}
});

// =====
// CONNECTION CLOSED
// =====

ws.addEventListener('close', (event) => {
    console.log('Disconnected from server');
    updateConnectionStatus('disconnected');

    // Attempt reconnection
    if (reconnectAttempts < maxReconnectAttempts) {
        reconnectAttempts++;
        const delay = Math.min(1000 * Math.pow(2, reconnectAttempts), 10000);
        console.log(`Reconnecting in ${delay}ms... (attempt ${reconnectAttempts})`);
        setTimeout(connect, delay);
    } else {
        addSystemMessage('Failed to reconnect. Please refresh the page.');
```

```
displayAvailableRooms(data.availableRooms);
}

function handleUsernameSet(data) {
  currentUsername = data.username;
  usernameSection.classList.add('hidden');
  chatSection.classList.remove('hidden');
  addSystemMessage(data.message);
}

function handleJoinedRoom(data) {
  currentRoom = data.room;
  currentRoomName.textContent = data.room;
  messagesContainer.innerHTML = "";
  addSystemMessage(data.message);
}

function handleLeftRoom(data) {
  currentRoom = null;
  currentRoomName.textContent = 'No room';
  membersList.innerHTML = "";
  addSystemMessage(data.message);
}

function handleChatMessage(data) {
  const isOwnMessage = data.username === currentUsername;
  addChatMessage(data.username, data.message, data.timestamp, isOwnMessage);
}

function handleUserJoined(data) {
  addSystemMessage(data.message);
}

function handleUserLeft(data) {
  addSystemMessage(data.message);
}

function handleRoomMembers(data) {
  membersList.innerHTML = "";
  data.members.forEach(member => {
    const li = document.createElement('li');
    li.textContent = member;
    if (member === currentUsername) {
      li.style.fontWeight = 'bold';
    }
    membersList.appendChild(li);
  });
});
```



```

    }

function handleRoomCreated(data) {
    addRoomToList(data.room);
    addSystemMessage(data.message);
}

function handlePrivateMessage(data) {
    addPrivateMessage(data.from, data.message, data.timestamp, false);
}

function handlePrivateMessageSent(data) {
    addPrivateMessage(data.to, data.message, data.timestamp, true);
}

function handleTyping(data) {
    // Show typing indicator
    console.log(`${data.username} is typing...`);
}

function handleError(data) {
    addSystemMessage(`Error: ${data.message}`, 'error');
}

// =====
// SEND FUNCTIONS - These trigger server's 'message' event
// =====

function sendMessage(type, payload) {
    if (ws && ws.readyState === WebSocket.OPEN) {
        const message = { type, ...payload };
        ws.send(JSON.stringify(message));
        console.log('Sent message:', message);
    } else {
        console.error('WebSocket is not connected');
        addSystemMessage('Not connected to server', 'error');
    }
}

function setUsername() {
    const username = usernameInput.value.trim();
    if (username) {
        sendMessage('set-username', { username });
    }
}

function joinRoom(roomName) {

```

```
sendMessage('join-room', { room: roomName });
}

function leaveRoom() {
  if (currentRoom) {
    sendMessage('leave-room', { room: currentRoom });
  }
}

function sendChatMessage() {
  const message = messageInput.value.trim();
  if (message && currentRoom) {
    sendMessage('chat', { message });
    messageInput.value = "";
  }
}

function createRoom() {
  const roomName = prompt('Enter new room name:');
  if (roomName) {
    sendMessage('create-room', { room: roomName });
  }
}

function sendPrivateMessage(toUsername) {
  const message = prompt(`Send private message to ${toUsername}:`);
  if (message) {
    sendMessage('private-message', { to: toUsername, message });
  }
}

// =====
// UI FUNCTIONS
// =====

function addChatMessage(username, message, timestamp, isOwn) {
  const messageDiv = document.createElement('div');
  messageDiv.className = `message ${isOwn ? 'own-message' : 'other-message'}`;

  const time = new Date(timestamp).toLocaleTimeString();

  messageDiv.innerHTML = `
    <div class="message-header">
      <span class="username">${username}</span>
      <span class="timestamp">${time}</span>
    </div>
    <div class="message-content">${escapeHtml(message)}</div>
  `;
}
```

```
`;  
  
messagesContainer.appendChild(messageDiv);  
messagesContainer.scrollTop = messagesContainer.scrollHeight;  
}  
  
function addSystemMessage(message, type = 'info') {  
  const messageDiv = document.createElement('div');  
  messageDiv.className = `system-message ${type}`;  
  messageDiv.textContent = message;  
  
  messagesContainer.appendChild(messageDiv);  
  messagesContainer.scrollTop = messagesContainer.scrollHeight;  
}  
  
function addPrivateMessage(user, message, timestamp, sent) {  
  const messageDiv = document.createElement('div');  
  messageDiv.className = 'private-message';  
  
  const time = new Date(timestamp).toLocaleTimeString();  
  const prefix = sent ? 'To' : 'From';  
  
  messageDiv.innerHTML = `  
    <div class="message-header">  
      <span class="username">${prefix}: ${user}</span>  
      <span class="timestamp">${time}</span>  
    </div>  
    <div class="message-content">${escapeHtml(message)}</div>  
  `;  
  
  messagesContainer.appendChild(messageDiv);  
  messagesContainer.scrollTop = messagesContainer.scrollHeight;  
}  
  
function displayAvailableRooms(rooms) {  
  roomsList.innerHTML = "";  
  rooms.forEach(room => {  
    addRoomToList(room);  
  });  
}  
  
function addRoomToList(roomName) {  
  const li = document.createElement('li');  
  li.textContent = roomName;  
  li.onclick = () => joinRoom(roomName);  
  roomsList.appendChild(li);  
}
```

```
function updateConnectionStatus(status) {  
  connectionStatus.className = `status-${status}`;  
  connectionStatus.textContent = status.charAt(0).toUpperCase() + status.slice(1);  
}
```

```
function escapeHtml(text) {  
  const div = document.createElement('div');  
  div.textContent = text;  
  return div.innerHTML;  
}
```

```
// =====  
// EVENT LISTENERS  
// =====
```

```
setUsernameBtn.addEventListener('click', setUsername);  
usernameInput.addEventListener('keypress', (e) => {  
  if (e.key === 'Enter') setUsername();  
});
```

```
sendBtn.addEventListener('click', sendChatMessage);  
messageInput.addEventListener('keypress', (e) => {  
  if (e.key === 'Enter' && !e.shiftKey) {  
    e.preventDefault();  
    sendChatMessage();  
  }  
});
```

```
createRoomBtn.addEventListener('click', createRoom);
```

```
// Typing indicator
```

```
let typingTimeout;  
messageInput.addEventListener('input', () => {  
  clearTimeout(typingTimeout);  
  sendMessage('typing', { isTyping: true });  
  
  typingTimeout = setTimeout(() => {  
    sendMessage('typing', { isTyping: false });  
  }, 1000);  
});
```

```
// Initialize connection on page load
```

```
connect();
```

3. HTML Interface (public/index.html)

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>WebSocket Chat Room</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <header>
      <h1>WebSocket Chat Room</h1>
      <div id="connection-status" class="status-disconnected">Disconnected</div>
    </header>

    <!-- Username Section -->
    <div id="username-section" class="section">
      <h2>Enter Your Username</h2>
      <div class="input-group">
        <input type="text" id="username-input" placeholder="Your username" maxlength="20">
        <button id="set-username-btn">Set Username</button>
      </div>
    </div>

    <!-- Chat Section -->
    <div id="chat-section" class="section hidden">
      <div class="chat-container">
        <!-- Sidebar -->
        <aside class="sidebar">
          <div class="sidebar-section">
            <h3>Rooms</h3>
            <button id="create-room-btn" class="btn-small">+ Create Room</button>
            <ul id="rooms-list"></ul>
          </div>

          <div class="sidebar-section">
            <h3>Members</h3>
            <ul id="members-list"></ul>
          </div>
        </aside>

        <!-- Main Chat Area -->
        <main class="chat-main">
          <div class="chat-header">
            <h2>Room: <span id="current-room-name">No room</span></h2>
          </div>
        </main>
      </div>
    </div>
  </div>
</body>
</html>
```

```
<div id="messages" class="messages-container"></div>

<div class="input-area">
  <textarea
    id="message-input"
    placeholder="Type your message..."
    rows="2"
  ></textarea>
  <button id="send-btn">Send</button>
</div>
</main>
</div>
</div>
</div>

<script src="app.js"></script>
</body>
</html>
```

4. Styling (public/style.css)

CSS

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}  
  
body {  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
  min-height: 100vh;  
  padding: 20px;  
}  
  
.container {  
  max-width: 1200px;  
  margin: 0 auto;  
  background: white;  
  border-radius: 10px;  
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);  
  overflow: hidden;  
}  
  
header {  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
  color: white;  
  padding: 20px;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}  
  
h1 {  
  font-size: 24px;  
}  
  
#connection-status {  
  padding: 5px 15px;  
  border-radius: 20px;  
  font-size: 12px;  
  font-weight: bold;  
  text-transform: uppercase;  
}  
  
.status-connected {  
  background: #10b981;  
  color: white;
```



```
}

.status-disconnected {
  background: #ef4444;
  color: white;
}

.status-error {
  background: #f59e0b;
  color: white;
}

.section {
  padding: 40px;
}

.hidden {
  display: none;
}

.input-group {
  display: flex;
  gap: 10px;
  max-width: 400px;
}

input[type="text"],
textarea {
  flex: 1;
  padding: 12px;
  border: 2px solid #e5e7eb;
  border-radius: 8px;
  font-size: 14px;
  font-family: inherit;
  transition: border-color 0.3s;
}

input[type="text"]:focus,
textarea:focus {
  outline: none;
  border-color: #667eea;
}

button {
  padding: 12px 24px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
```

```
border: none;
border-radius: 8px;
font-size: 14px;
font-weight: bold;
cursor: pointer;
transition: transform 0.2s, box-shadow 0.2s;
}

button:hover {
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}

button:active {
  transform: translateY(0);
}

.btn-small {
  padding: 8px 16px;
  font-size: 12px;
  width: 100%;
  margin-bottom: 10px;
}

.chat-container {
  display: grid;
  grid-template-columns: 250px 1fr;
  height: 600px;
}

.sidebar {
  background: #f9fafb;
  border-right: 1px solid #e5e7eb;
  padding: 20px;
  overflow-y: auto;
}

.sidebar-section {
  margin-bottom: 30px;
}

.sidebar-section h3 {
  font-size: 14px;
  color: #6b7280;
  text-transform: uppercase;
  margin-bottom: 10px;
}
```

```
.sidebar-section ul {  
  list-style: none;  
}
```

```
.sidebar-section li {  
  padding: 10px;  
  margin-bottom: 5px;  
  background: white;  
  border-radius: 6px;  
  cursor: pointer;  
  transition: background 0.2s;  
}
```

```
.sidebar-section li:hover {  
  background: #e5e7eb;  
}
```

```
.chat-main {  
  display: flex;  
  flex-direction: column;  
}
```

```
.chat-header {  
  padding: 20px;  
  border-bottom: 1px solid #e5e7eb;  
  background: #f9fafb;  
}
```

```
.chat-header h2 {  
  font-size: 18px;  
  color: #374151;  
}
```

```
.messages-container {  
  flex: 1;  
  padding: 20px;  
  overflow-y: auto;  
  background: white;  
}
```

```
.message {  
  margin-bottom: 15px;  
  padding: 12px;  
  border-radius: 8px;  
  max-width: 70%;  
  animation: slideIn 0.3s ease;
```

```
}

@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.own-message {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  margin-left: auto;
}

.other-message {
  background: #f3f4f6;
  color: #1f2937;
}

.private-message {
  background: #fef3c7;
  border-left: 4px solid #f59e0b;
  margin-bottom: 15px;
  padding: 12px;
  border-radius: 8px;
  max-width: 70%;
}

.message-header {
  display: flex;
  justify-content: space-between;
  margin-bottom: 5px;
  font-size: 12px;
}

.username {
  font-weight: bold;
}

.timestamp {
  opacity: 0.7;
}
```

```
.message-content {  
  word-wrap: break-word;  
  line-height: 1.5;  
}
```

```
.system-message {  
  text-align: center;  
  padding: 8px;  
  margin: 10px 0;  
  border-radius: 6px;  
  font-size: 13px;  
  background: #e0e7ff;  
  color: #4338ca;  
}
```

```
.system-message.error {  
  background: #fee2e2;  
  color: #991b1b;  
}
```

```
.input-area {  
  padding: 20px;  
  border-top: 1px solid #e5e7eb;  
  display: flex;  
  gap: 10px;  
  background: #f9fafb;  
}
```

```
.input-area textarea {  
  flex: 1;  
  resize: none;  
}
```

```
.input-area button {  
  align-self: flex-end;  
}
```

/* Scrollbar Styling */

```
::-webkit-scrollbar {  
  width: 8px;  
}
```

```
::-webkit-scrollbar-track {  
  background: #f1f1f1;  
}
```

```
::-webkit-scrollbar-thumb {  
  background: #667eea;  
  border-radius: 4px;  
}  
  
::-webkit-scrollbar-thumb:hover {  
  background: #764ba2;  
}  
  
/* Responsive Design */  
@media (max-width: 768px) {  
  .chat-container {  
    grid-template-columns: 1fr;  
    height: auto;  
  }  
  
  .sidebar {  
    border-right: none;  
    border-bottom: 1px solid #e5e7eb;  
    max-height: 200px;  
  }  
  
  .messages-container {  
    height: 400px;  
  }  
}
```

5. Package.json

```
json
```

```
{
  "name": "websocket-chat-room",
  "version": "1.0.0",
  "description": "Real-time chat room with WebSocket",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "ws": "^8.14.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

How to Run

1. Install Dependencies

```
bash
npm install
```

2. Start Server

```
bash
npm start
```

3. Open Multiple Browser Windows

Open `http://localhost:3000` in multiple browser windows to test the chat.

How It All Works Together

The Flow Explained

```
CLIENT SENDS MESSAGE:
1. User types "Hello" in input field
2. User clicks Send button
```

3. JavaScript calls: `sendMessage()`
4. Function calls: `sendMessage('chat', { message: 'Hello' })`
5. This does: `ws.send(JSON.stringify({ type: 'chat', message: 'Hello' }))`
6. WebSocket protocol automatically wraps this in a frame
7. Frame is sent over TCP connection to server

SERVER RECEIVES MESSAGE:

8. Server's WebSocket library receives the frame
9. WebSocket library automatically triggers: `ws.on('message', (data) => {...})`
10. Server parses JSON: `const message = JSON.parse(data.toString())`
11. Server checks `message.type: 'chat'`
12. Server routes to: `handleChatMessage(ws, message)`
13. Server broadcasts to all clients in the room

OTHER CLIENTS RECEIVE:

14. Each client's `ws.addEventListener('message', ...)` fires
15. Each client parses the JSON
16. Each client checks `type: 'chat'`
17. Each client calls: `handleChatMessage(data)`
18. Message appears in chat UI

Key Insight

You don't manually create different event types! WebSocket only has ONE `'message'` event. What you DO is:

1. **Structure your messages with a `type` field**
2. **Parse the type on the receiving end**
3. **Route to appropriate handler based on type**

This is why we use JSON:

```
javascript

// Instead of trying to create different events (impossible)
ws.sendCustomEvent('chat', data); // ❌ This doesn't exist!

// We structure our data with types
ws.send(JSON.stringify({ type: 'chat', data })); // ✅ This works!
```


Message Types Used

Type	Direction	Purpose
set-username	Client → Server	Set user's display name
join-room	Client → Server	Join a chat room
leave-room	Client → Server	Leave current room
chat	Client → Server	Send chat message
create-room	Client → Server	Create new room
private-message	Client → Server	Send DM to user
typing	Client → Server	Typing indicator
connection	Server → Client	Welcome message
username-set	Server → Client	Confirm username
joined-room	Server → Client	Confirm room join
left-room	Server → Client	Confirm room leave
user-joined	Server → Broadcast	User joined room
user-left	Server → Broadcast	User left room
room-members	Server → Client	List of room members
room-created	Server → Broadcast	New room created
error	Server → Client	Error message

Testing the Chat

Test Scenario 1: Basic Chat

1. Open browser window 1, set username "Alice"
2. Open browser window 2, set username "Bob"
3. Both join "general" room
4. Alice sends "Hello Bob!"
5. Bob sees the message and replies "Hi Alice!"

Test Scenario 2: Multiple Rooms

1. Alice joins "tech" room
2. Bob stays in "general" room
3. Messages sent in "tech" only visible to users in "tech"
4. Alice switches to "general" to chat with Bob

Test Scenario 3: Room Creation

1. Alice clicks "Create Room"
 2. Enters "project-alpha"
 3. New room appears in both Alice and Bob's room list
 4. Bob clicks "project-alpha" to join
-

Summary

The key takeaway: **WebSocket protocol only has one `message` event.** The "type" system is something WE implement in our application layer by structuring our JSON messages with a `type` field. This allows us to have different "logical" message types (chat, join, leave, etc.) all flowing through the same WebSocket `message` event.